Name	Definition	Used in	Purpose	Syntax	Example
sorted()	It is given to sort given tuple either ascending or descending	i] tuple	To sort tuple	sorted (tuple-name, reverse = True / False)	a = (10, 2, 5, 100) print (sorted (a, reverse = True) # (100, 10, 5, 2)
find()	It can find the index position of particular character in string	i] string	To find index	string-name . find ('element')	a = "hello" a. find ('o') print (a) # 5
starts-with()	It is used to check the string starts with specific character or not.	i] string	To check the character start with.	string-name . startswith ('char')	a = "hello" a . startswith ('h') print (a) # True
ends-with()	It is used to check the string ~~starts~~ ends with specific character or not.	i) string	To check the character ends with	string-name . endswith ('char')	a = "hello" a . endswith ('o') print (a) # True
replace()	It is used to replace single character or word from a string	i] string	To replace character	string-name . replace (existing char, replaceable char)	a = "hello" print (a . replace ('l', 'D # he DDo

Name	Definition	Used in	Purpose	Syntax	Example
insert()	It is used to insert the element into the sequence at particular index position.	i] array ii] list	To insert element at particular index position.	sequence-name . insert (index, element)	a = [10, 30, 60, 70, 80] a . insert (2, 100) print (a) # [10, 30, 60, 100, 70, 80]
append()	It is used to insert element into sequence at end.	i] array ii] list	To insert element at end.	sequence - name . append (element)	a = [1, 2, 3, 4] a . append (200) print (a) # [1, 2, 3, 4, 200]
clear()	It is used to clear all elements from sequence.	i] array ii] list iii] set iv] dictionary	To clear all elements.	sequence - name . clear ()	a = {10, 20, 70.5, 80} a . clear () print (a) # {}
sort()	It is used to sort sequence either ascending or descending	i] array ii] list	To sort ascending or descending.	sequence - name . sort (reverse = True / False)	a = [10, 20, 30, 40] a . sort (reverse = True) print (a) # [40, 30, 20, 10]
extend()	It is used for to insert more than one element or sub-sequence at end of sequence.	i] array ii] list	To insert sub-sequence at end.	sequence - name . extend (element / sub-seqⁿ)	a = [10, 20, 30] a . extend ([200, 300]) print (a) # [10, 20, 30, 200, 300]
index()	It can find index position of particular element in sequence.	i] string ii] tuple	To find index position.	sequence - name . index (element)	a = (10, 20, "hello", 30) a . index ("hello") print (a) # 2

Name	Definition	Used in	Purpose	Example	Syntax
len()	It is used to count the length of sequence	i] string ii] array iii] list iv] tuple v] set vi] dictionary	To count the length.	Ex:- a = [10, 20, 30, 70, 80] print(len(a)) # 4	len(sequence-name)
remove()	It is used to remove element from sequence.	i] array ii] list iii] set	To remove element	Ex:- a = [100, 20, 200] a.remove(20) print(a) # [100, 200]	sequence-name.remove (element)
pop()	It is used to remove element from sequence by using index.	i] array ii] list iii] set	To remove element by index.	Ex:- z = [10, 20, 30, 60] z.pop(1) print(z) # [10, 60]	sequence-name.pop (index)
count()	It is used to count occurance of element into given sequence.	i] array ii] list iii] tuple	To count occurance of a element.	Ex:- x = [10, 30, 20, 10, 50] z = x.count(10) print(z) # 2	sequence-name.count (element)
min()	It is used to find minimum element in sequence.	i] array ii] list iii] tuple	To find minimum element	Ex:- a = [10, 20, 30, 40] x = min(a) print(x) # 10	min(sequence-name)
max()	It is used to find maximum element in sequence.	i] array ii] list iii] tuple	To find maximum element	Ex:- c = [10, 20, 100, 60] z = max(c) print(z) # 100	max(sequence-name)

②	Implicit	Explicit
1.	Compiler converts one datatype into another	1. Programmer converts one datatype into another.
2.	There is no data loss.	2. Chances of data loss.
3.	short → int → long int double ← float	3. double → float → long int short ← int
4.	There is no specific syntax.	4. int (variable)
5.	int a = 40; float b = a;	5. float a = 40.20; int b = int (a)

③	Local variables	Global variables
1.	Variables which are defined inside function definition	1. Variables which are defined outside the function definition.
2.	Life is till the compiler is inside function	2. Life is till the execution of program.
3.	Scope is limited inside the function definition	3. Scope is through out the program.

1] POP & OOP

→

	POP	OOP
1]	POP stands for Procedure Oriented Programming	1] OOP stands for Object Oriented Programming
2]	It focuses on functions	2] It focuses on objects.
name) no) 3]	It follows top-down approach.	3] It follows bottom-up approach.
4]	It does not support the features of OOP's	4] It supports all the features of POP.
5]	Communication is done between functions	5] Communication is done between objects.
6]	Inheritance property is not used.	6] Inheritance property is used.
7]	It does not support data hiding.	7] Encapsulation is used to hide the data.
8]	Ex:- C, FORTRAN, etc.	8] Ex:- Python, CPP, Java C#, etc.

①	break	continue
1.	The remaining part of current iteration & all the next iterations are skipped.	1. The remaining part of current iterations is skipped & all the next are continued.
2.	It terminates the loop	2. It does not terminates the loop.
3.	It is used in loops & switch statements.	3. It is used only in loops
4.	Syntax:- break	4. Syntax:- continue

5.	Ex:-
	for i in range(0,5,1):
	if(i==2):
	break
	print(i)
	
	O/P:- 0
	1
		5. for i in range(1,6,1):
		if(i==2):
		continue
		print(i)
		O/P:- 1
		3
		4
		5